

Technical Report for the IMU Odometry Challenge

Single-Head IMU Velocity Estimation Across Platforms via a Multi-Scale, Embodiment-Aware Encoder

Kaggle team name: Team - SPARO

Jiwon Choi¹, Hogyun Kim¹, Jungwoo Lee¹, Geonmo Yang¹, and Seunghee Yun¹

Abstract—Inertial measurement units (IMUs) are mounted on almost every robot, which makes them a natural basis for one inertial model shared across platforms. However, the same IMU signal describes different motion on a car, a quadruped, a drone and a handheld device, and the platform is not given at test time. We propose a single network with one velocity head for all four platforms, built on two principles. *Multi-Scale*: the encoder reads the IMU from tens of milliseconds to the whole trajectory, using a residual stem, S4D state-space layers and a bidirectional GRU. *Embodiment*: a head infers the platform from the IMU alone, and its posterior conditions the encoder, the head and the output scale. We further adapt the network to unseen recording sessions by self-distillation on the unlabelled test inputs, towards a teacher refined by strapdown IMU integration. Our experiments showed that embodiment conditioning is necessary, since a model trained without one platform failed on it with $3.8\text{--}27\times$ its in-distribution error. Our submission achieved a TartanIMU Score of 0.18405 over the 89 test sequences (ATE₂₀ 0.498 m, AVE 0.147 m/s, RTE 0.680 m) and 0.23110 on the public leaderboard.

I. INTRODUCTION

An inertial measurement unit (IMU) describes the ego-motion of a robot regardless of lighting, texture or weather, and learned inertial odometry (IO) now regresses velocity directly from IMU windows for pedestrians [2], [3], [4] and robots [5], [6]. Most of these models, however, are trained for one platform. Tartan IMU [1] moved towards a foundation model by pre-training across platforms, and this challenge asks the next question: can one model with one set of weights serve four embodiments (car, quadruped, drone and handheld) whose identity is withheld at test time?

A single model across platforms faces two challenges. First, the informative structure of an IMU lives at different time scales. Footfalls and impacts last tens of milliseconds, gait and rotor vibration fill a second, and velocity itself, which an IMU does not measure, has to be carried across many seconds. Second, the platforms differ by an order of magnitude in speed, and the same IMU pattern means different motion on each. The drones even report in two different IMU frame conventions. In addition, training and test are different recording sessions: our model fitted the labelled sessions, which it was trained on, to a score of 0.050 (local metric), but reached 0.184 on the unseen test sessions.

¹The authors are with the Department of Electrical and Computer Engineering, Inha University, Incheon, Republic of Korea. [jiwon2, hg.kim, pihsdneirf, ygm7422, shyun28]@inha.edu

Younggun Cho: Faculty advisor of Team - SPARO

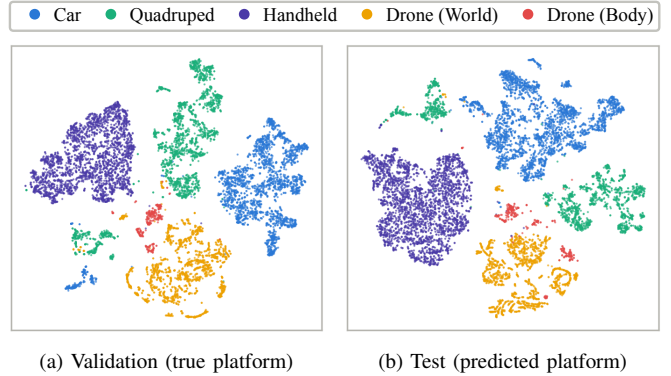


Fig. 1. t-SNE of window-encoder features. (a) Validation windows coloured by their true platform. (b) Test windows coloured by the platform the network infers, whose per-platform trajectory and window counts match the official test breakdown. In both splits, the drones whose IMU reports in the world frame and in the body frame form separate clusters (for test, the two groups are inferred from raw IMU statistics).

To address these challenges, we propose a single network built on two principles, *Multi-Scale* and *Embodiment*. Multi-Scale matches the encoder to the time scales of the IMU, and Embodiment lets one velocity head serve every platform without a platform label. Our main contributions are as follows:

- **Multi-Scale Encoder:** We design an encoder that matches its operator to each time scale of the IMU. Its window stage, *Motion-SSM*, combines a residual stem, S4D [9] layers and order-preserving tokens, and it was the best of twelve encoders we screened.
- **Embodiment-Aware Single Head:** The network infers a platform posterior from the IMU alone and conditions on it, so one velocity head serves every platform (Fig. 1). A leave-one-platform-out study shows why: a shared model does not transfer to an unseen embodiment.
- **Adaptation to the Test Sessions:** We train the network by self-distillation on the unlabelled test inputs, towards a teacher refined by strapdown IMU integration. This gave most of the improvement from 0.285 to our submitted 0.23110 on the public leaderboard.
- **Comprehensive Analysis:** We report per-platform, per-sequence, generalization and failure-mode results, derive the one post-processing rule that the metric allows, and record the ideas that did not help.

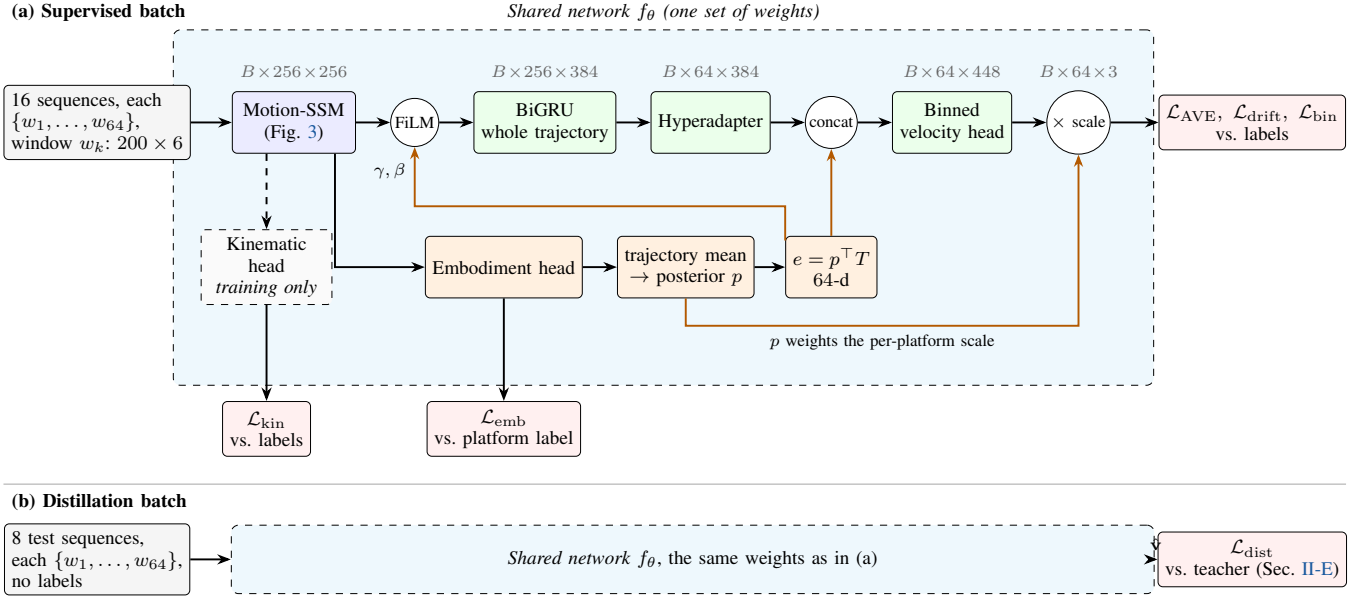


Fig. 2. Architecture and training. The shaded box is the shared network f_θ , shown in full in (a) and collapsed in (b); red boxes are loss terms and grey labels are tensor sizes (B sequences of 64 windows, four tokens per window). The embodiment posterior conditions the network through FiLM before the BiGRU, concatenation before the head and a per-platform output scale. Each training step uses (a) labelled sequences, which train every head against ground truth, the platform label entering only $\mathcal{L}_{emb}$, and (b) unlabelled test sequences, whose velocities are pulled towards the teacher. The dashed head exists only in training.

II. METHOD

We propose a single network f_θ of 5 416 375 parameters, as illustrated in Fig. 2. A trajectory is a sequence of one-second windows $\{w_1, \dots, w_L\}$, where each window $w_k \in \mathbb{R}^{200 \times 6}$ holds 200 samples of the three-axis gyroscope and accelerometer. The network estimates the mean body-frame velocity of every window as follows:

$$\{\hat{\mathbf{v}}_k\}_{k=1}^L = f_\theta(\{w_k\}_{k=1}^L), \quad \hat{\mathbf{v}}_k \in \mathbb{R}^3. \quad (1)$$

Following the Multi-Scale principle, the encoder builds the representation from local IMU events to the whole trajectory (Sec. II-A). Following the Embodiment principle, the network infers the platform and conditions a single shared head on it (Sec. II-B). The network is trained on the labelled data (Sec. II-C–II-D), adapted to the test sessions by self-distillation (Sec. II-E), and followed by one metric-derived rule at inference (Sec. II-F).

A. Multi-Scale Encoder

An IMU window mixes events whose durations differ by two orders of magnitude. Therefore, we build the representation at three time scales and use at each scale the operator suited to it.

1) *Local Features*: Short events, such as footfalls, impacts, and rotor and road vibration, span tens of milliseconds. A ResNet-style stem [7] extracts them. It consists of a stride-2 convolution with kernel 7 and two residual blocks of kernel-3 convolutions with batch normalisation, and maps the 200×6 window to 100 steps of 320 channels.

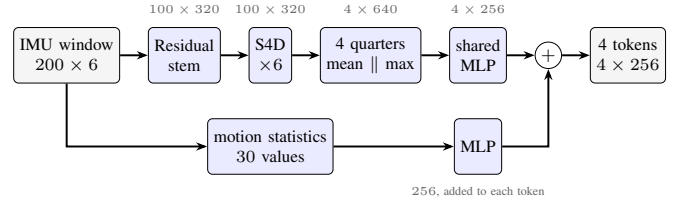


Fig. 3. The window encoder *Motion-SSM*: local features from a residual stem, window-level features from S4D layers, four order-preserving tokens per window, and a motion-statistics summary added to each token.

2) *Window Features*: Within one second, gait rhythm, vibration and slow drift in acceleration and tilt overlap. Rather than fixing which frequency bands to look at, we let six S4D layers [8], [9] learn them. Each channel c is filtered by a kernel that spans all 100 steps as follows:

$$K_c(t) = \text{Re} \sum_{n=1}^{64} C_{c,n} e^{(-\alpha_{c,n} + i\omega_{c,n}) t \Delta_c}, \quad (2)$$

where α , ω , C and Δ are the learned decay, frequency, weight and step size of 64 damped oscillations. Hence one layer sees the whole second with a learned spectral selectivity. To keep the order of events inside the second, we split the output into four quarters. Each quarter is mean- and max-pooled and mapped by a shared MLP to a 256-dimensional token. In addition, a motion-statistics branch embeds per-axis summary statistics of the raw IMU (mean, standard deviation, RMS, half-to-half change and mean derivative) and adds them to every token. Together, these layers form the window encoder *Motion-SSM* (Fig. 3).

3) *Trajectory Features*: A single window rarely determines velocity, since the IMU does not measure it and its level must be carried across windows. We therefore lay out the tokens of all windows in time order and run a two-layer bidirectional GRU (BiGRU, hidden size 192) over them. At inference, the BiGRU covers the whole trajectory in one pass. The four tokens of each window are then averaged back to one feature per window, and a hyperadapter adds four low-rank residual bases (rank 16), mixed by a router that reads the trajectory-mean feature.

B. Embodiment-Aware Conditioning

The platform is never given, yet the same IMU pattern means different motion on a car and on a quadruped. Since the encoder tokens already separate the platforms (Fig. 1), an embodiment head g scores every token z_j of a trajectory over the four platforms. The platform posterior p and the platform embedding e are computed as follows:

$$p = \text{softmax}\left(\frac{1}{|J|} \sum_{j \in J} g(z_j)\right), \quad e = p^\top T, \quad (3)$$

where J is the set of tokens of the trajectory and $T \in \mathbb{R}^{4 \times 64}$ is a learned table. The embedding conditions the network in three places. First, FiLM [10] rescales and shifts the encoder tokens before the BiGRU, i.e., $z \leftarrow z \odot (1 + \gamma(e)) + \beta(e)$. Second, e is concatenated to the features before the head. Third, p weights a learned per-platform, per-axis output scale $S \in \mathbb{R}^{4 \times 3}$. The platform label is used only as the training target of the embodiment head, so the conditioning is soft and learned jointly, and the same weights serve every embodiment.

All platforms share one velocity head. For each axis a , it predicts a distribution $q_{k,a}$ over 64 bins with centres c_b in normalised units, and the velocity is obtained as follows:

$$\hat{v}_{k,a} = (p^\top S)_a \sum_b q_{k,a,b} c_b. \quad (4)$$

C. Data Processing

We used the challenge’s canonical non-overlapping one-second windows at 200 Hz without re-windowing, resampling or filtering. Inputs were standardised with fixed per-channel statistics stored in the checkpoint, and the model was trained on sequences of 64 consecutive windows with random start offsets. We restricted augmentation to transformations whose effect on the label is known exactly: time scaling of drone trajectories (log-uniform in $[0.85, 2.0]$, with the label scaled accordingly), sub-segment crops of at least 8 windows from short trajectories, and random IMU biases, white noise and a 2% accelerometer scale error, which leave the label unchanged.

As listed in Table I, the network was trained on all 475 labelled trajectories, including the official validation split. The 89 test trajectories contributed only their unlabelled IMU, through the self-distillation of Sec. II-E. No data beyond the challenge data and no externally pretrained weights were used.

TABLE I
DATASETS USED FOR MODEL DEVELOPMENT AND TRAINING.

Dataset	Type	Usage
Challenge train set (395 traj.)	Official	Supervised training
Challenge validation set (80 traj.)	Official	Supervised training; monitoring
Challenge test inputs (89 traj.)	Official, unlabelled	Self-distillation towards our own predictions
Warm-start checkpoint	Own	Initialisation: soup of 11 of our networks, trained on the 475 labelled traj.

D. Training Objective and Optimization

1) *Overall Loss*: Each step draws a supervised batch of 16 labelled sequences and a distillation batch of 8 test sequences, passes both through the same network, and minimises the following loss:

$$\mathcal{L} = \mathcal{L}_{\text{AVE}} + 0.175 \mathcal{L}_{\text{drift}} + 0.1 \mathcal{L}_{\text{emb}} + 0.1 \mathcal{L}_{\text{bin}} + \mathcal{L}_{\text{kin}} + 0.3 \mathcal{L}_{\text{dist}}, \quad (5)$$

where $\mathcal{L}_{\text{AVE}} = \text{mean}_k \|\hat{\mathbf{v}}_k - \mathbf{v}_k\|$ is the AVE metric itself for the true velocity $\mathbf{v}_k$, and $\mathcal{L}_{\text{dist}}$ is defined in Sec. II-E.

2) *Drift Loss*: To charge only the error that ATE₂₀ measures, we rotate both velocity sequences to the world frame with the ground-truth attitude R_k and integrate them, i.e., $\hat{\mathbf{p}}_i = \sum_{k \leq i} R_k \hat{\mathbf{v}}_k \delta t$ and likewise $\mathbf{p}_i$. Each sequence is cut into segments $\mathcal{S}$ whose length, chosen from $\{4, 8, 16, 32, 48, 64\}$ windows, is closest in log scale to 20 m of travel. The drift loss is then defined as follows:

$$\mathcal{L}_{\text{drift}} = \text{mean}_{\mathcal{S}} \left(\frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \|\text{sg}(R_{\mathcal{S}}^* \tilde{\mathbf{p}}_i - \tilde{\mathbf{p}}_i)\|^2 \right)^{1/2}, \quad (6)$$

where $\tilde{\mathbf{p}}_i = \mathbf{p}_i - \bar{\mathbf{p}}_{\mathcal{S}}$ is the centred path, $R_{\mathcal{S}}^*$ is the Procrustes [17] rotation that best aligns $\tilde{\mathbf{p}}$ to $\tilde{\mathbf{p}}$, and $\text{sg}(\cdot)$ stops the gradient. The alignment absorbs the rigid error that the metric forgives.

3) *Auxiliary Losses*: $\mathcal{L}_{\text{emb}}$ is the cross-entropy of the embodiment head against the platform label, which is the only place the label is used. $\mathcal{L}_{\text{bin}}$ is the KL divergence from a Gaussian soft label around each target component to the 64-bin distribution of the head. $\mathcal{L}_{\text{kin}}$ trains an auxiliary head on the window encoder towards kinematic targets available only in training, i.e., sub-step velocity, velocity change and relative rotation within the window, plus consistency with $\hat{\mathbf{v}}_k$ (weights in Table X).

4) *Optimization*: We used AdamW (learning rate 3×10^{-4} , weight decay 10^{-4}) with cosine annealing and kept an exponential moving average (EMA) of the weights. The submitted run took 66 epochs (4.1 h on one NVIDIA A100) and was warm-started from an earlier checkpoint of the same architecture. That checkpoint is itself the product of the self-distillation generations of Sec. II-E, so the submitted run is the last of seven warm-started runs (roughly 30–35 GPU-hours in total). We submitted the EMA weights of the final epoch (seed 43). The full configuration is in Table X.

E. Self-Distillation on the Test Inputs

Supervised training alone did not close the session gap on the test drones, which carry about half of the score. Since the test inputs are available, we additionally train the network to reproduce, on the test IMU, the velocities of a *teacher*, i.e., a prediction file made by our own models and fixed before each training run. This is a form of self-training and knowledge distillation [11], [12], [13], [14] in the transductive setting of test-time adaptation [15], [16].

1) *Distillation Loss*: The distillation batch contributes the following term:

$$\mathcal{L}_{\text{dist}} = \text{mean}_k \|\hat{\mathbf{v}}_k - \mathbf{v}_k^{\text{teacher}}\|, \quad (7)$$

which is computed through the same head as $\mathcal{L}_{\text{AVE}}$. Each new student then provides the predictions for the next teacher, so the gains compound over generations (Table VI).

2) *Teacher Construction*: We improved the teacher in four steps:

- 1) *Averaged predictions*. The first teacher averaged the test predictions of two of our models.
- 2) *Physics smoother*. For the eleven test drones whose IMU reports in the body frame, strapdown integration provided information independent of the network (see below).
- 3) *Speed correction*. A drone whose physics-estimated mean speed exceeded the network's by more than 30% and was above 2 m/s was rescaled to the physics speed (thresholds set on labelled drones).
- 4) *Re-derived teacher*. The eleven drones were re-derived by steps 2 and 3 from the improved student, and three pairs among them, which the IMU identifies as two sensors on the same flight, were fused. All other windows kept the previous teacher, divided by a global 1.01.

The eleven drones were identified by labelling train and validation drones with their measured frame convention and assigning test drones by nearest neighbour on raw IMU statistics. All refinement used only the test IMU and our predictions.

3) *Physics Smoother*: Over segments of 64 windows, the smoother estimates $\vartheta = (\mathbf{r}_0, \mathbf{b}_g, \mathbf{b}_a, \mathbf{u}_0, s)$, i.e., the initial attitude, the gyroscope and accelerometer biases, the initial world-frame velocity and a gyroscope scale. Given the gyroscope $\boldsymbol{\omega}_t$ and accelerometer $\mathbf{a}_t$, the state is integrated as follows:

$$\begin{aligned} R_{t+1} &= R_t \text{Exp}(s(\boldsymbol{\omega}_t - \mathbf{b}_g) \delta t), \\ \mathbf{u}_{t+1} &= \mathbf{u}_t + (R_t(\mathbf{a}_t - \mathbf{b}_a) + \mathbf{g})\delta t, \end{aligned} \quad (8)$$

where $\mathbf{g}$ is gravity. The window means $\bar{\mathbf{v}}_k(\vartheta)$ of the body-frame velocity $R_t^\top \mathbf{u}_t$ are then fitted to the student's predictions as follows:

$$\min_{\vartheta} \sum_k \rho(\bar{\mathbf{v}}_k(\vartheta) - \hat{\mathbf{v}}_k) + \lambda_g \|\mathbf{b}_g\|^2 + \lambda_a \|\mathbf{b}_a\|^2 + \lambda_s (s-1)^2, \quad (9)$$

where ρ is a robust soft- L_1 loss and $\lambda_{g,a,s}$ are prior weights. Segments whose fit residual exceeds a threshold keep the

TABLE II
MODEL COMPLEXITY AND RUNTIME STATISTICS.

Property	Value
Number of parameters	5 416 375
Training hardware	1 × NVIDIA A100 80 GB
Training time	4.1 h (submitted run)
Inference hardware	1 × NVIDIA A100 80 GB
Inference speed (Hz)	5 604 windows/s
Peak memory	0.36 GiB (GPU)
Online / causal	No (whole-trajectory)

TABLE III
OVERALL CHALLENGE PERFORMANCE ON THE **FINAL (PRIVATE)**
LEADERBOARD. TARTANIMU SCORE
= 0.6 AVE/0.7356 + 0.4 ATE₂₀/3.1160. RTE IS DEFINED IN THE
CAPTION OF TABLE V.

Method	Score ↓	ATE ₂₀ ↓	AVE ↓	RTE ↓
TartanIMU Baseline	—	—	—	—
Ours	0.18405	0.49766	0.14733	0.68021

network prediction. In short, the network supplies the velocity level that the IMU cannot observe, and the IMU supplies the dynamics within each segment.

F. Inference and Low-Speed Rule

At inference, each test trajectory takes one forward pass. The only per-trajectory quantities are the embodiment posterior (Eq. (3)) and the router input of the hyperadapter, both averaged over the trajectory, and inputs are normalised with fixed training statistics. Then one global rule is applied as follows:

$$\hat{\mathbf{v}} \leftarrow \hat{\mathbf{v}} \cdot \left[1 - b \left(1 - \min \left(\frac{\|\hat{\mathbf{v}}\|}{s_0}, 1 \right) \right) \right], \quad (10)$$

where $b = 0.03$ and $s_0 = 0.5$ m/s were chosen on the validation split (not held out for this model). Thus, velocities below 0.5 m/s are shortened by at most 3% and faster ones are unchanged. The rule follows from the metric. The score charges 0.8156 per m/s of AVE but only 0.1284 per metre of ATE₂₀, so post-processing can pay only if it leaves AVE intact. At low speed, the direction of a one-second mean velocity is uncertain. Under a directional error θ , the AVE-minimising estimate is $v_{\text{gt}} \cos \theta$, which is shorter than the truth, and a slightly short path is what the per-segment SE(3) alignment forgives.

G. Model Complexity and Runtime

Table II summarises the cost. The network processed all 30 644 test windows in 5.5 s, far faster than real time. However, the BiGRU and the trajectory-mean posterior read the whole trajectory, so the system runs offline rather than causally.

III. DEVELOPMENT EXPERIMENTS

We developed the network stage by stage, following the path of the signal, and Table VIII in Appendix E records every comparison. We used three evaluation settings. The

TABLE IV
PERFORMANCE BY PLATFORM. RTE IS DEFINED IN THE CAPTION OF
TABLE V.

Platform	ATE ₂₀ ↓	AVE ↓	RTE ↓
Wheeled / Car	0.34113	0.06675	0.24902
Handheld / Human	0.54523	0.06708	0.31299
Legged / Quadruped	0.34564	0.07813	0.37684
Aerial / Drone	0.75862	0.37736	1.78198
Average	0.49766	0.14733	0.68021

early screening split (S) was later found to overlap training and therefore supports relative comparison only. The official validation split (V) was held out until the final recipe trained on it. All validation numbers use our local implementation of the metric. Finally, public-leaderboard differences below about 0.002 were within run-to-run variation.

1) *Window Encoder*: Among twelve window encoders at matched capacity, only the S4D encoder beat the ResNet1D baseline. Attention over the 200 samples, explicit frequency-band splits and yaw canonicalisation all performed worse. Splitting each window into four order-preserving tokens gave the largest architectural gain, whereas removing the window hierarchy and pre-training (the organisers’ 40 Hz model and JEPA) both hurt.

2) *Embodiment and Sequence Model*: The embodiment head identified all 80 validation trajectories. Hard-routed expert GRUs helped on S, but adding experts to the encoder hurt, so we kept soft conditioning. The recurrent cell did not matter (LSTM performed the same as GRU). In contrast, replacing sequential state with a global operator (Transformer or S4D) cost 0.035–0.043, since useful context ends at about ± 8 windows (± 16 for drones).

3) *Training and Output*: Before distillation, the largest gains came from aligning the drift loss with ATE₂₀ (Eq. (6)) and from short-crop sampling of drone flights. Self-distillation then gave the largest gain overall (Table VI). In contrast, augmentations that re-pose existing data (yaw and mirror), weight soups and test-time training [15] all hurt. Every global output transform except Eq. (10) was also rejected. Smoothing removed real signal, since the ground truth changes by 73% of the mean speed between adjacent windows, and sharpening had nothing to recover.

IV. EXPERIMENTS

A. Official Challenge Performance

1) *Metrics*: The TartanIMU Score (Table III) is macro-averaged over the four platforms, and the all-zero submission scores 1.000. AVE is the mean per-window velocity error, and ATE₂₀ is the error over 20m segments after SE(3) alignment. RTE [18], as used in AirIO [6], is reported for analysis only. It is the RMSE of the 5 s relative displacement and is insensitive to accumulated offset.

2) *Results*: All numbers in Tables III, IV and V were produced by the official scoring service, [https://huggingface.co/spaces/Tartan-IMU/imu_odometry_challenge_](https://huggingface.co/spaces/Tartan-IMU/imu_odometry_challenge_scoring)

[scoring](https://huggingface.co/spaces/Tartan-IMU/imu_odometry_challenge_scoring), from the submission CSV with md5 = **4ef8fbce3e8f0610b180b89262cf60f1**. The same file scored 0.23110 on the public leaderboard.

B. Per-Platform Results

Since the score is macro-averaged, each platform carries a fixed quarter of the weight. Nevertheless, drones contributed 55% of our score, and their AVE was five times that of the other platforms (Table IV). This reflects speed rather than a drone-specific failure. Drone windows averaged 1.51 m/s of predicted speed against 0.30 m/s for the rest, and normalised by speed, the four platforms lay within a few points of one another (about 25% relative error). Across the 89 sequences, AVE correlated with mean speed at $r = 0.88$.

C. Per-Sequence Results

The full per-sequence results are given in Table V and Fig. 4 in the appendix. The errors were concentrated. Sequences 29 and 56 were outliers on every metric, with AVE near 2 m/s against a median of 0.15 m/s. They are two IMUs on one racing flight whose peak speed, estimated from its IMU by strapdown integration, is 10–11.5 m/s, well beyond the fastest labelled drone (5.8 m/s). Thus the failure is speed extrapolation rather than a sensor defect. Moreover, ATE₂₀ correlated with AVE across sequences ($r = 0.76$). The three same-flight pairs scored almost identically, but this is partly induced by their fused teacher targets.

D. Ablation Studies

Table VI lists the development history of the submitted model in order. Each row is one model warm-started from the row above and may bundle several changes. Self-distillation accounted for most of the gain, in particular the physics-refined teacher (−0.0228) and the fast-flight speed correction (−0.0147). The low-speed rule left validation AVE unchanged, improved ATE₂₀ on all four platforms, and its validation change (−0.00014) matched its test change exactly.

E. Generalization Analysis

1) *Across Platforms*: To test whether a shared inertial model learns embodiment-independent physics, we trained on three platforms and evaluated on the fourth (Table VII). Every held-out platform collapsed, to 3.8–27× its in-distribution AVE, and the handheld case was the worst. Therefore, the shared model works because it has seen every platform, not because its representation transfers to an unseen one. This is the reason for the Embodiment principle. Note that the in-distribution reference comes from a related configuration rather than a matched control, but the ratios far exceed such a mismatch.

2) *Platform Inference*: The embodiment posterior was one-hot on every test trajectory, and the inferred platforms matched the official per-platform counts exactly (18 car, 16 quadruped, 45 drone and 10 handheld trajectories). On validation, a linear probe recovered the platform with 0.81 balanced accuracy from raw IMU statistics, but with 0.99

TABLE VI

DEVELOPMENT HISTORY OF THE SUBMITTED MODEL, ONE TRAINED MODEL PER ROW (TARTANIMU SCORE, LOWER IS BETTER; –: NOT SCORED ON THE 89-SEQUENCE SERVICE; ‡: SCORED WITH FOUR-OFFSET TEMPORAL TEST-TIME AUGMENTATION, WHILE ALL LATER ROWS AND THE SUBMISSION USE ONE WHOLE-TRAJECTORY PASS).

Model	Public	89-seq.
Warm-start checkpoint (weight soup), no distillation	0.28521 [‡]	–
+ distillation, averaged-prediction teacher	0.27856 [‡]	–
+ physics-smoother teacher (11 drones)	0.25580 [‡]	–
+ gravity handling in augmentation fixed; whole-trajectory inference	0.25231	0.19475
+ speed-corrected fast flight (teacher gen. 4)	0.23764	0.18905
+ EMA weights, drone time scaling to 2.0	0.23679	0.18833
+ teacher re-derived from the student	0.23171	–
+ velocity-bin auxiliary loss (network)	0.23129	0.18419
+ low-speed rule (submitted)	0.23110	0.18405

TABLE VII

LEAVE-ONE-PLATFORM-OUT: TRAIN ON THREE PLATFORMS, EVALUATE ON THE FOURTH (80 EPOCHS, MEAN OF THE LAST FIVE).

Held out	AVE	ATE ₂₀	In-distr. AVE	Ratio
Quadruped	0.328	2.53	≈0.087	3.8×
Car	0.515	4.06	≈0.095	5.4×
Drone	1.469	2.55	≈0.283	5.2×
Handheld	2.307	30.14	≈0.086	27×

from the window encoder. The two drone IMU conventions were also separated in the encoder (0.99), whereas the four-way embodiment head merged them into one class. Accordingly, the network gave one intermediate response to a heading rotation for both conventions, and augmentation that corrected this response did not improve the public score.

3) *Across Sessions*: The remaining gap lies between recording sessions. Since validation is both in-session and, in the final recipe, trained on, it was a poor selection signal. Across six checkpoints, the best validation score belonged to one of the worst public scores (Table IX, $r = -0.54$).

F. Failure-Mode Analysis

The largest failures occurred in fast drone flight, above all the out-of-range flight 29/56, where the residual is per-window scatter rather than a constant rotation. Oracle substitutions on the validation split (trained on, local metric) placed the remaining error in direction. The true direction improved the score by 0.0147 and the true speed by 0.0103, whereas the true per-sequence mean speed made it worse, since our speed is already calibrated to within 1.3%. A global rotation recovered nothing. In addition, the system is offline because it reads the whole trajectory.

V. INSIGHTS AND LESSONS LEARNED

1) *A Single Model Needs to Know the Embodiment*: One set of weights and one head served all four platforms, but

the shared model did not transfer to an unseen embodiment (Table VII). For an IMU foundation model, this suggests that covering embodiments and conditioning on them matter more than a platform-agnostic representation.

2) *Multi-Scale Structure Matters*: Learned oscillatory kernels within a second, order-preserving tokens within the window and sequential state over about ± 8 windows all helped. In contrast, attention over samples and global operators over the sequence did not.

3) *Unlabelled Test Inputs Help under Session Shift*: Self-distillation gave more than all architectural changes combined, because it moves information that no single network produces, such as averaged predictions and strapdown physics, into one set of weights. In contrast, test-time training with a self-supervised loss hurt. For regression, the teacher has to carry information, not only a consistency objective.

4) *The Remaining Error Is Per-Window Direction*: The model was already calibrated in scale, rotation and per-sequence speed, so only a better model can reduce the remaining directional noise.

5) *The Metric Decides Which Post-Processing Can Help*: AVE costs 6.4 times more per unit than ATE₂₀, which explains almost every post-processing idea we discarded. Only the validation change of a small, global, AVE-neutral transform transferred to the test set.

A. Feedback to the Organizers

Because AVE costs 6.4 times more per unit than ATE₂₀ and is absolute, fast drone flight carried 55% of our score. A speed- or per-platform-normalised AVE would weight the embodiments more evenly. In addition, documenting the two IMU frame conventions would let participants model the platforms rather than reverse-engineer the data.

VI. CONCLUSION

We proposed a single network with one velocity head for four platforms, built on two principles. The Multi-Scale encoder reads the IMU from local events to the whole trajectory, and the Embodiment conditioning lets one head serve every platform without a platform label. Self-distillation on the test inputs further adapted the network to unseen sessions, and the submission achieved 0.18405 on the 89 test sequences and 0.23110 on the public leaderboard. Our results suggest that a single model is feasible but must know the embodiment. Future work includes a causal variant for online use and a label-free estimate of the IMU frame convention.

ACKNOWLEDGMENT

We thank the TartanIMU and Super Odometry organisers for the dataset, the scoring service and the re-execution requirement.

REFERENCES

- [1] S. Zhao, S. Zhou, R. Blanchard, Y. Qiu, W. Wang, and S. Scherer, "Tartan IMU: A Light Foundation Model for Inertial Positioning in Robotics," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 22520–22529.

- [2] C. Chen, X. Lu, A. Markham, and N. Trigoni, "IONet: Learning to Cure the Curse of Drift in Inertial Odometry," in *Proc. AAAI Conf. on Artificial Intelligence*, 2018.
- [3] S. Herath, H. Yan, and Y. Furukawa, "RoNIN: Robust Neural Inertial Navigation in the Wild: Benchmark, Evaluations, & New Methods," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2020.
- [4] W. Liu, D. Caruso, E. Ilg, J. Dong, A. I. Mourikis, K. Daniilidis, V. Kumar, and J. Engel, "TLIO: Tight Learned Inertial Odometry," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020.
- [5] Y. Qiu, C. Wang, C. Xu, Y. Chen, X. Zhou, Y. Xia, and S. Scherer, "AirIMU: Learning Uncertainty Propagation for Inertial Odometry," arXiv:2310.04874, 2023.
- [6] Y. Qiu, C. Xu, Y. Chen, S. Zhao, J. Geng, and S. Scherer, "AirIO: Learning Inertial Odometry with Enhanced IMU Feature Observability," *IEEE Robotics and Automation Letters*, 2025.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [8] A. Gu, K. Goel, and C. Ré, "Efficiently Modeling Long Sequences with Structured State Spaces," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2022.
- [9] A. Gu, A. Gupta, K. Goel, and C. Ré, "On the Parameterization and Initialization of Diagonal State Space Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [10] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual Reasoning with a General Conditioning Layer," in *Proc. AAAI Conf. on Artificial Intelligence*, 2018.
- [11] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015.
- [12] T. Furlanello, Z. Lipton, M. Tschannen, L. Itti, and A. Anandkumar, "Born-Again Neural Networks," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2018.
- [13] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-Training with Noisy Student Improves ImageNet Classification," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [14] A. Tarvainen and H. Valpola, "Mean Teachers Are Better Role Models: Weight-Averaged Consistency Targets Improve Semi-Supervised Deep Learning Results," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [15] Y. Sun, X. Wang, Z. Liu, J. Miller, A. Efros, and M. Hardt, "Test-Time Training with Self-Supervision for Generalization under Distribution Shifts," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2020.
- [16] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully Test-Time Adaptation by Entropy Minimization," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2021.
- [17] S. Umeyama, "Least-Squares Estimation of Transformation Parameters Between Two Point Patterns," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [18] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2012, pp. 573–580.

TABLE V

PER-SEQUENCE CHALLENGE RESULTS OVER THE 89 TEST SEQUENCES. ATE, AVE AND RTE ARE PRODUCED BY THE OFFICIAL PER-SEQUENCE SCORING SERVICE. **RTE** (RELATIVE TRAJECTORY ERROR, M) FOLLOWS STURM *et al.* [18] AS USED IN AIRIO [6]: OVER SLIDING WINDOWS OF DURATION Δt , IT IS THE RMSE OF THE DIFFERENCE BETWEEN PREDICTED AND GROUND-TRUTH RELATIVE DISPLACEMENT,

$$\text{RTE} = \sqrt{\frac{1}{n} \sum_i \|(\hat{\mathbf{p}}_{i+\Delta t} - \hat{\mathbf{p}}_i) - (\mathbf{p}_{i+\Delta t} - \mathbf{p}_i)\|^2}. \text{ UNLIKE ATE IT MEASURES LOCAL DRIFT AND IS INSENSITIVE TO ACCUMULATED GLOBAL OFFSET. } \Delta t = 5 \text{ S FOR ALL PLATFORMS.}$$

Seq. ID	ATE ₂₀ ↓	AVE ↓	RTE ↓	Seq. ID	ATE ₂₀ ↓	AVE ↓	RTE ↓	Seq. ID	ATE ₂₀ ↓	AVE ↓	RTE ↓
1	0.89658	0.36574	1.93642	31	0.68244	0.35945	1.53073	61	0.77614	0.24656	1.24502
2	0.63195	0.28423	1.27866	32	0.32268	0.07709	0.22198	62	0.10675	0.01814	0.06190
3	0.70871	0.21818	1.01881	33	0.18974	0.04983	0.19818	63	0.55899	0.08915	0.40553
4	0.38655	0.25931	0.72426	34	0.45091	0.18151	0.70744	64	0.44294	0.07759	0.30221
5	0.58699	0.24787	1.06782	35	1.03925	0.19311	1.03777	65	0.17514	0.04449	0.15039
6	0.66980	0.08935	0.76223	36	0.99170	0.36934	1.83384	66	1.45692	0.25607	1.83453
7	0.65333	0.22672	0.93754	37	0.49894	0.61327	2.94748	67	0.14344	0.03320	0.17101
8	0.40591	0.04837	0.21930	38	0.50317	0.59443	2.90006	68	0.31168	0.03696	0.14583
9	0.44524	0.22066	0.65831	39	0.28325	0.16100	0.46110	69	0.70119	0.10912	0.48450
10	1.77890	0.13552	0.90682	40	0.86084	0.38888	1.87538	70	0.80177	0.09173	0.39920
11	1.24250	0.76782	2.30819	41	1.06527	0.41235	2.22180	71	0.11699	0.05365	0.14218
12	0.26060	0.18780	0.76388	42	0.58502	0.09485	0.39988	72	0.82459	0.45665	1.53644
13	0.24688	0.04185	0.22458	43	0.75710	0.28268	1.14727	73	0.48276	0.10736	0.41286
14	0.65719	0.27069	1.26802	44	0.25054	0.05619	0.32205	74	0.32608	0.22014	0.55727
15	1.08796	0.08578	0.45132	45	0.08100	0.05178	0.12246	75	0.69778	0.29588	1.26824
16	0.10492	0.03269	0.20594	46	0.24526	0.05475	0.20422	76	0.42341	0.08776	0.27741
17	0.85568	0.31623	1.73891	47	0.73287	0.33264	1.42708	77	1.20149	0.57765	2.25256
18	1.36625	0.41908	2.10861	48	0.22109	0.01824	0.10668	78	0.10861	0.02513	0.12233
19	0.47708	0.25073	0.92674	49	0.80752	0.26196	1.40511	79	0.26238	0.15169	0.62475
20	0.12526	0.05258	0.35130	50	0.23889	0.11970	0.41471	80	0.22275	0.13930	0.60927
21	0.75597	0.19615	1.21756	51	0.04139	0.02624	0.06017	81	0.44609	0.11681	0.35574
22	0.78666	0.35430	1.52071	52	0.12424	0.05237	0.15400	82	0.81507	0.16815	0.97476
23	1.00201	0.28165	1.92618	53	0.40456	0.27980	0.52789	83	0.45435	0.07604	0.36839
24	0.56087	0.05863	0.25220	54	0.14290	0.05079	0.18145	84	0.60138	0.07179	0.38554
25	0.11144	0.03318	0.12050	55	0.27803	0.17329	0.81811	85	0.11666	0.03829	0.14693
26	0.35958	0.07350	0.22334	56	2.25051	2.06494	10.97715	86	0.43133	0.03850	0.22239
27	0.44068	0.07367	0.30029	57	0.94126	0.41192	2.18726	87	0.28821	0.15292	0.57783
28	0.45778	0.15831	0.76129	58	0.28074	0.05468	0.26012	88	1.02318	0.36110	2.11129
29	2.26076	1.92179	10.53774	59	0.21524	0.06978	0.19251	89	0.09243	0.02192	0.06890
30	0.88665	0.34328	1.83953	60	0.15341	0.06716	0.21455				

APPENDIX

A. Per-Sequence Results (Full Table)

Table V lists the official scoring service’s output for all 89 test sequences, from the submission CSV with md5 = 4ef8fbce3e8f0610b180b89262cf60f1; trends are discussed in Sec. IV-C.

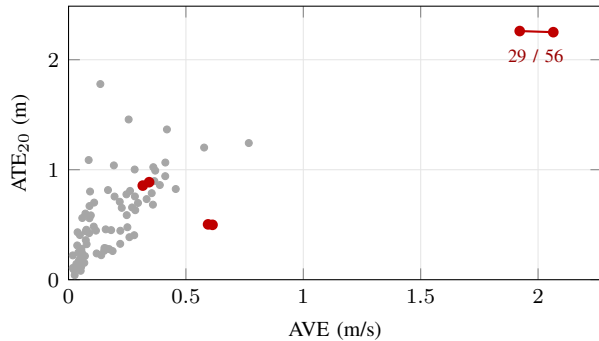


Fig. 4. ATE₂₀ against AVE for the 89 test sequences ($r = 0.76$). Red pairs joined by a line are two IMUs on the same drone flight (Sec. II-E); the outliers 29 and 56 are one such flight.

Fig. 4 plots the table: ATE₂₀ against AVE, with the three same-flight pairs joined. The racing flight 29/56 sits alone

in the upper right; the other 87 sequences spread along one band ($r = 0.76$), so no second failure cluster exists.

B. Reproducibility and Submission Artifacts

- **Team and submission.** Kaggle team Team - SPARO (no former names); members Jiwon Choi, Hogyun Kim, Jungwoo Lee, Geonmo Yang and Seunghee Yun. Kaggle submission ID 56400889, uploaded as `lss_b03.csv`, identical to `submission.csv` in the repository (MD5 4ef8fbce3e8f0610b180b89262cf60f1), one of our final selected submissions; public leaderboard 0.23110, scoring service 0.18405 (89 sequences).
- **Code.** <https://huggingface.co/jivvon2/tartanimu-sparo>, commit 60a1975.
- **Checkpoint.** `ckpt/model.pt`, MD5 8e550e1c669a0aeba879delaebc1a3e7 (EMA weights).
- **Configuration.** Stored inside the checkpoint and rebuilt by the loader; the full training configuration is Table X.
- **Environment.** Python 3.12.13, PyTorch 2.11.0, CUDA 13.0; dependencies pinned in `code/requirements.txt`.
- **Hardware and time.** Training: one NVIDIA A100 80 GB, 4.1 h for the submitted run (all seven warm-started runs of Table VI took roughly 30–35 GPU-

hours). Inference: one A100, 5.5 s for all 30 644 test windows, 0.36 GiB peak memory; CPU also supported.

- **Seeds.** Seed 43; the reported score is a single training run. Public-score differences between runs of one recipe are of the order of 0.002 (Sec. III).
- **External data.** None: no external dataset, synthetic data or pretrained weights beyond our own checkpoints. The released TartanIMU checkpoint was evaluated during development (Table VIII) but is not in the ancestry of the submitted weights.
- **Validation numbers.** All validation numbers are computed with our local implementation of the metric. The submitted model and its warm-start ancestors were trained on train and validation, so for them the validation split is not held out; we have no train-only twin of the submitted model. Numbers marked V in Table VIII were held out for the models they compare.
- **Constants and how they were chosen.** Low-speed rule $b = 0.03$, $s_0 = 0.5$ m/s: validation split (not held out). Teacher speed correction, 30% and 2 m/s: labelled drones of the train and validation splits. Teacher global factor 1/1.01: validation split. Distillation weight $10\times$ on the 345 windows of the eleven drones: set by hand, with alternative weightings ($0.3\times$ and a further $10\times$) compared on the public leaderboard. Initial per-platform output scales of the warm-start checkpoint (car, quadruped and handheld 0.98, drone 1.02): public-leaderboard feedback; they are trainable and were changed by training.
- **Teacher.** Teacher files are fixed prediction CSVs. The physics smoother is a non-linear least-squares fit that reproduces to about 2×10^{-3} across environments rather than bit-exactly; this affects only rebuilding a teacher, not the submitted network.
- **Timing.** The weights were published at 2026-09-21 05:09 UTC, about five hours after the 20 September 23:55 UTC deadline, because we read the weights deadline as the report deadline. The published checkpoint reproduces the CSV submitted to Kaggle before the close (to 4.4×10^{-8}), so it is the model that produced the submission.

Compliance declaration.

- 1) **Single model with one shared set of weights?** Yes. Every submitted prediction comes from one checkpoint, loaded once. The embodiment is inferred inside the network; there are no separately trained per-platform experts.
- 2) **Platform classification, routing or specialisation at inference?** Yes, inside the network only, and computed from the IMU alone. The embodiment head reads the encoder tokens and averages its logits over the trajectory into a four-way posterior (Eq. (3)). Its embedding FiLM-modulates the encoder tokens before the BiGRU and is concatenated to the features before the velocity head, and the posterior weights a per-platform, per-axis output scale (4×3 parameters), all trained jointly

as one set of weights. The posterior is one-hot on all 89 test trajectories, so in effect the network selects one row of the scale table and one embedding per trajectory. No platform label is read. The test-drone grouping of Sec. II-E exists only in training.

- 3) **Ensembling or TTA?** No prediction ensembling and no test-time augmentation: each test trajectory takes exactly one forward pass over the whole trajectory, with no overlapping chunks, shifted windows, offsets or rotated views. Weight averaging is used, which yields one set of weights: (i) the submitted weights are the exponential moving average (decay 0.999) of a single training run; (ii) the first ancestor of the warm-start chain is a weight-space average (soup) of 11 of our own networks of the same architecture, trained on the 475 labelled trajectories, followed by BatchNorm recalibration on train windows. Every later run warm-starts from the previous one (Table VI). Separately, the self-distillation teacher descends from the averaged predictions of two of our networks (first generation only); this average is a training target, not part of the submitted system.
- 4) **Avoiding test-set leakage.** No test label is used anywhere. The unlabelled test IMU is used during training, as follows: (i) *which*: the IMU of all 89 test trajectories (30 644 windows); (ii) *how*: as inputs of the self-distillation batch, whose targets are a teacher file made of our own predictions (Sec. II-E); for eleven drones, the teacher was refined from the test IMU alone by the physics smoother, a speed correction and fusion of recordings that the IMU identifies as the same flight, and these drones were found by nearest neighbour on raw test-IMU statistics; (iii) *when*: training only, over seven successive warm-started runs. No statistics are fitted on the test set, and input normalisation uses fixed statistics from training data. Model and checkpoint selection used the validation split and public-leaderboard feedback on our own submissions. Public-leaderboard feedback also set the initial per-platform output scales of the warm-start checkpoint (see Constants above). Throughout development we also inspected the per-sequence results of the official scoring service for our own submissions, for analysis only; no value was fitted to them.
- 5) **Development versus submitted system.** The network is the trained model unchanged; the submitted system adds only the low-speed rule of Eq. (10). Augmentations and post-processing explored during development (Sec. III, Appendix E) are not part of it.

C. Final Predictions

The final predictions are the CSV submitted to Kaggle as `lss_b03.csv` (submission.csv in the repository, MD5 4ef8fbce3e8f0610b180b89262cf60f1, public 0.23110): <https://huggingface.co/jivvon2/tartanimu-sparo/blob/main/submission.csv>. It has the columns `window_id`, `vx`, `vy`, `vz` (mean

body-frame velocity in m/s) and 30644 rows plus the header; every id of `index/test_windows.csv` appears exactly once and all values are finite. No window needs special handling. The only post-processing after the network output is Eq. (10); there is no test-time augmentation, temporal smoothing, per-sequence rescaling, ensembling or soup.

D. Model and Inference Code

The repository above contains the checkpoint, the inference entry point `code/predict.py`, the model and data packages, and the pinned requirements. From the repository root:

```
pip install -r code/requirements.txt
python code/predict.py \
  --data-root <DATA> \
  --checkpoint ckpt/model.pt \
  --out submission.csv

<DATA> must contain test/,
index/test_windows.csv and
sample_submission.csv, against which the row
set is validated before writing. The low-speed rule is applied
by default (--shrink-b 0.03 --shrink-s0 0.5).
The command reproduces the submitted CSV to  $4.4 \times 10^{-8}$ 
per component (float32 rounding) on a single GPU with less
than 1 GB of memory, or on CPU with --device cpu.
```

E. Development Record

Table VIII is the full development record summarised in Sec. III, and Table IX supports the selection analysis of Sec. IV-E.

How much of a teacher a student absorbs. Two teachers were themselves submitted before being distilled, so the transfer can be measured. The first teacher, an average of two models’ predictions, scored 0.27768 from a 0.28521 starting model; its student reached 0.27856, i.e. 88% of the gain inside one network. The physics-smoother teacher scored 0.25511; its student reached 0.25580, 97% of the gain. Distillation is therefore close to lossless here, which is what makes it the vehicle for non-submittable gains.

Screening instruments. Split S was built early for fast screening and later found to share 79% of its evaluation trajectories (88% for drone) with training, so its numbers support ranking only. Official validation V was then used until the final recipe trained on all labelled data. Neither ranks the final candidates, whose public scores differ by less than 0.002.

F. Implementation Details

Table X collects every setting of the submitted training run in one place.

TABLE VIII
DEVELOPMENT RECORD BY PIPELINE STAGE.

Change	Eval.	Result
<i>Window encoder</i>		
ResNet1D window encoder (baseline)	S	0.2265
S4D window encoder	S	0.2181
Transformer over 200 samples	S	0.2795
Frequency-band split	S	0.2412
Yaw canonicalisation	S	0.4254
1 → 4 sub-window tokens	S	0.1965 → 0.1860
Hierarchy-free trunk vs. hierarchical	V	0.2054 vs. 0.1783
Organisers’ 40 Hz checkpoint, frozen	P	0.500
JEPA pre-training vs. from scratch	S	0.2248 vs. 0.2181
<i>Embodiment</i>		
Held-out platform, AVE vs. in-distribution	–	3.8–27×
Embodiment head , trajectories correct	V	80/80
Hard-routed expert GRUs (not submitted)	S	–0.0104
Expert encoders and GRUs	S	+0.0108
<i>Sequence model and head</i>		
BiGRU / LSTM	S	0.1226 / 0.1225
Transformer / S4D	S	0.1580 / 0.1658
Recurrence masked over padding	V	0.1845 → 0.1826
Training sequence 64 → 128 windows	V	0.1826 → 0.1876
Velocity-bin auxiliary loss	P	0.23171 → 0.23129 [†]
Heterogeneous MoE vs. BiGRU	P	0.23372 vs. 0.23171
<i>Training</i>		
Drift loss aligned with ATE₂₀	P	0.33402 → 0.32187
Short-crop sampling	P	–0.0267
Yaw augmentation, all platforms	S	0.2767 → 0.3429
Mirror augmentation	V	0.1826 → 0.1926
Self-distillation , teacher generations	P	0.28521 [‡] → 0.23171
Weight soup of loss-weight variants	P	0.26241 vs. 0.23129
Test-time training (TTT) at inference	V	worse at every setting
<i>Output (change in validation score)</i>		
Speed scale × 1.01	Δ	+0.00334
Temporal box smoothing ($w = 2$)	Δ	+0.22445
Global rotation (0.165°)	Δ	–0.00002
Low-speed rule , Eq. (10)	Δ	–0.00014

S: screening split (overlaps training; relative only); V: official validation split; P: public leaderboard; Δ: change on V; †: within run-to-run variation; ‡: four-offset temporal TTA.

TABLE IX
VALIDATION (TRAINED-ON) SCORE AGAINST PUBLIC SCORE FOR SIX CHECKPOINTS OF THE FINAL FAMILY.

Checkpoint	Validation	Public
Pair-fusion weight 0.65	0.04267	0.23439
Teacher generation 4	0.04793	0.23764
Drone speed-scale augmentation	0.04900	0.23147
Submitted network	0.05023	0.23129
Bin-KL weight 0.05	0.05118	0.23138
Teacher re-derived from the student	0.05146	0.23171

TABLE X
FULL CONFIGURATION OF THE SUBMITTED RUN.

Item	Value
<i>Input and data</i>	
Window	200×6 (1 s at 200 Hz), canonical index
Training sequence	64 windows, random offset; short crops ≥ 8
Batch	16 labelled + 8 test (distillation) sequences
Normalisation	fixed per-channel statistics (in checkpoint)
<i>Network (5 416 375 parameters)</i>	
Stem	conv $k=7$, stride 2; 2 residual blocks, 320 ch.
S4D	6 layers, 64 modes per channel, 100 steps
Tokens	4 per window, mean max, shared MLP, 256-d
Motion statistics	30 values, MLP, added to each token
Sequence model	BiGRU, 2 layers, hidden 192 per direction
Hyperadapter	4 low-rank bases, rank 16, trajectory router
Embodiment	4 classes, table $T \in \mathbb{R}^{4 \times 64}$; FiLM, concat, 4×3 scale
Velocity head	64 bins per axis, expectation
<i>Loss weights (Eq. (5))</i>	
AVE / drift / emb. / bin / dist.	1 / 0.175 / 0.1 / 0.1 / 0.3
Kinematic	sub-step 0.2, Δv 0.1, rotation 0.05, consistency 0.1
Drift spans	$\{4, 8, 16, 32, 48, 64\}$ windows, nearest to 20 m
<i>Augmentation</i>	
Drone time scaling	log-uniform [0.85, 2.0]
Bias (acc. / gyro)	0.1 / 0.002 (input units)
White noise (acc. / gyro)	0.02 / 0.002
Acc. scale error	2%
<i>Optimisation</i>	
Optimiser	AdamW, lr 3×10^{-4} , wd 10^{-4}
Schedule	cosine to 9×10^{-6} , 66 epochs
Epoch	9 920 training sequences
Clipping / EMA	1.0 / 0.999 (EMA submitted)
Initialisation	own checkpoint, velocity head reinitialised
Seed	43
<i>Distillation</i>	
Test sampling	514 chunks of 64 windows; 11 drone chunks $\times 10$
Smoother	64-window segments, 50 Hz, soft- L_1 , 4 headings
<i>Inference</i>	
Pass	whole trajectory, one forward pass
Low-speed rule	$b = 0.03$, $s_0 = 0.5$ m/s